

BLOQUE 1: INVESTIGACIÓN

¿Qué es Docker y para qué sirve?

Docker es una plataforma de código abierto diseñada para facilitar la creación, despliegue y gestión de aplicaciones dentro de contenedores. Estos contenedores funcionan como pequeñas «cajas virtuales» que incluyen todo lo necesario para que una aplicación corra correctamente: el código, las librerías, dependencias, herramientas de sistema y archivos de configuración, independientemente del sistema operativo sobre el que se ejecute el host

Para que sirve para la estandarización y simplificación del proceso de mover software entre distintos entornos. El clásico problema de «en mi PC funciona, pero en producción no» queda resuelto: lo que va dentro del contenedor se comporta igual en desarrollo, pruebas o producción, ya sea en un portátil, un servidor físico o en la nube.

Diferencia entre imagen y contenedor

La imagen Docker es el punto de partida para crear cualquier contenedor. Se trata de un paquete ejecutable, creado habitualmente a partir de un Dockerfile, en el que se incluyen:

Código fuente de la aplicación

Dependencias y librerías de sistema

Herramientas adicionales y archivos de configuración

Las imágenes Docker son inmutables y utilizan un sistema de capas: cada vez que se modifica la imagen, se añade una nueva capa, de modo que es fácil hacer rollback a versiones previas o reutilizar partes comunes entre imágenes distintas.

Cuando se ejecuta una imagen, Docker crea automáticamente una «capa de contenedor» que recoge los cambios realizados en tiempo de ejecución (como lectura/escritura de archivos o actualizaciones temporales), sin alterar la imagen original.

Un contenedor es, básicamente, una instancia viva de una imagen Docker. Una vez arrancado, se comporta como un entorno independiente, donde la aplicación corre

exactamente como se definió en la imagen, pero además, puede almacenar datos y configuraciones que sólo existen mientras dura su ejecución.

¿Qué es Docker Compose?

Cuando una aplicación requiere varios servicios (como una base de datos, un servidor web y una caché), Docker Compose entra en juego. Permite definir, mediante un archivo YAML, cómo deben lanzarse y comunicarse los diferentes contenedores de tu stack.

Ventajas clave de Docker Compose:

- **Simplifica la orquestación** de proyectos complejos, evitando tener que arrancar cada contenedor manualmente.
- **Permite replicar entornos de producción** de manera idéntica en local, minimizando errores en el despliegue.
- **Facilita la documentación y versionado** de la infraestructura de servicios.

Con un simple comando **docker-compose up** puedes arrancar todos los servicios definidos bajo demanda.

4. ¿Cómo se comunican los contenedores?

La forma más sencilla de conectar contenedores de Docker es colocándolos en la misma red. Cuando los contenedores están en la misma red, pueden comunicarse entre sí utilizando sus nombres de contenedor o direcciones IP.

Para conectar contenedores en la misma red, puede utilizar la bandera `--network` al iniciar un nuevo contenedor:

Ahora, los dos contenedores pueden comunicarse entre sí utilizando sus nombres de contenedor (por ejemplo, `container1`, `container2`) o direcciones IP.

Conectando contenedores entre redes

En algunos casos, es posible que necesite conectar contenedores entre diferentes redes. Esto se puede lograr utilizando la bandera `--link` o creando una superposición de red (network overlay).

Utilizando la bandera `--link`:

Create two networks

```
docker network create network1
```

```
docker network create network2
```

Start a container in network1

```
docker run -d --name container1 --network network1 nginx
```

Start a container in network2 and link it to container1

```
docker run -d --name container2 --network network2 --link container1 nginx
```

Ahora, container2 puede acceder a container1 utilizando el nombre de host container1.

Conectando contenedores entre redes utilizando una superposición de red:

5. ¿Por qué ocurre **ECONNREFUSED db:3306**?

El error **ECONNREFUSED** en Docker al intentar conectarse a la base de datos MySQL con el nombre de servicio "db" y el puerto 3306 puede deberse a varias razones. Una de las causas más comunes es que el nombre de la base de datos no coincida con el nombre de servicio definido en el archivo `.env` de Docker. Esto puede ocurrir si hay un error de escritura en el archivo `.env`, como un error de tipo de dato incorrecto o un nombre de base de datos incorrecto.

Para solucionar este problema, es importante verificar que el nombre de la base de datos en el archivo .env sea correcto y coincida con el nombre de servicio definido en el archivo Docker Compose. También es recomendable asegurarse de que las credenciales de conexión en el archivo .env sean correctas y coincidan con las que se especifican en el archivo Docker Compose.

6. ¿Qué es un volumen y por qué es importante?

Los volúmenes de Docker son esencialmente directorios o archivos que se montan dentro de un contenedor, permitiendo que los datos se almacenen y accedan por el contenedor. Los volúmenes pueden ser creados y gestionados por Docker, o pueden ser montados mediante enlace (bind-mounted) desde el sistema de archivos del host.

Los volúmenes ofrecen varias ventajas sobre el almacenamiento de datos dentro de la capa de escritura del contenedor:

Persistencia de Datos: Los volúmenes permiten que los datos persistan incluso después de que un contenedor se detiene o elimina, asegurando que los datos importantes no se pierdan.

Compartir Datos: Los volúmenes pueden compartirse entre múltiples contenedores, permitiendo el intercambio de datos y la colaboración entre aplicaciones.

Rendimiento: Los volúmenes pueden proporcionar un mejor rendimiento que el almacenamiento de datos en la capa de escritura del contenedor, especialmente para cargas de trabajo intensivas en E/S.

Copia de Seguridad y Restauración: Los volúmenes pueden respaldarse, restaurarse o migrarse fácilmente a otros entornos, lo que facilita la gestión y protección de datos importantes.

7. ¿Cómo usarías IA para resolver errores?

Lo primero sería identificar los log que se están presentando, copiar el log y preguntarle a la ia que significa el error y como se soluciona,

Otra forma es pedirle que revise mi dockerfile o Docker-compose si falta alguna instrucción o el orden de las capas es incorrecto, o explicarle a la ia la estructura de mi contenedores si tengo varios y hay algún problema de conexión entre ellos que me identifique porque esta fallando

BLOQUE 2: EJERCICIO PRÁCTICO

1. Crear estructura del proyecto
2. Implementar backend en Node.js
3. Crear Dockerfile
4. Crear docker-compose.yml con backend y MySQL
5. Ejecutar: docker-compose up --build

Validaciones:

- API funcionando en localhost:3000
- Conexión a base de datos
- Logs sin errores

The screenshot shows a VS Code editor with a project named 'MODULO 11'. The Explorer sidebar on the left shows the file structure: 'src' (containing 'index.js'), '.dockerignore', '.env', 'docker-compose.yml', 'Dockerfile', and 'package.json'. The main editor window displays the 'index.js' file with the following code:

```
1 const express = require("express");
2 const mysql = require("mysql2/promise");
3 const swaggerJsdoc = require("swagger-jsdoc");
4 const swaggerUi = require("swagger-ui-express");
5
6 const app = express();
7 app.use(express.json());
8
9 const PORT = process.env.PORT || 3000;
10
11 // Swagger config
12 const swaggerSpec = swaggerJsdoc({
13   definition: {
14     openapi: "3.0.0",
15     info: {
```

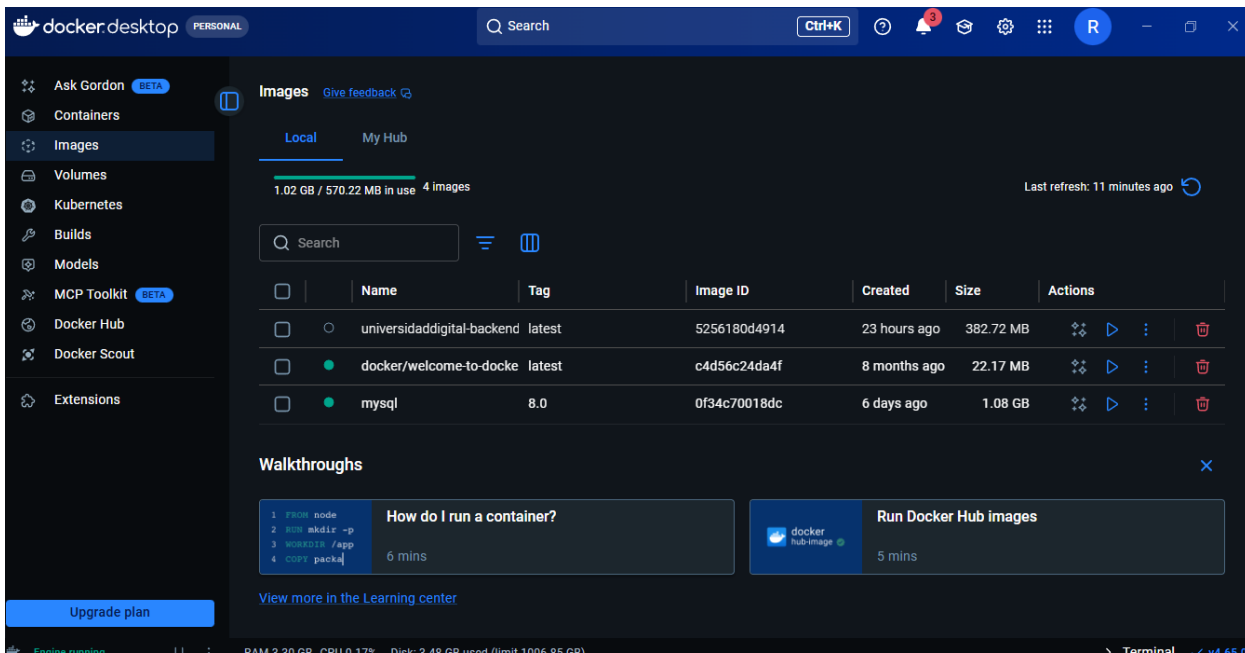
Below the code, the 'TERMINAL' tab is active, showing the command prompt output:

```
(c) Microsoft Corporation. Todos los derechos reservados.
C:\Users\USER\Documents\dev-senior\modulo 11>cd src
C:\Users\USER\Documents\dev-senior\modulo 11>docker ps
```

The output of the 'docker ps' command is shown in a table format:

CONTAINER ID	IMAGE	COMMAND	NAMES	CREATED	STATUS	PORTS
e2381009831b	modulo11-backend	"docker-entrypoint.s..."	modulo11-backend-1	9 minutes ago	Up 9 minutes	0.0.0.
0:3000->3000/tcp, [::]:3000->3000/tcp						
abc9e463f2e1	mysql:8.0	"docker-entrypoint.s..."	modulo11-mysql-1	9 minutes ago	Up 9 minutes (healthy)	0.0.0.
0:3307->3306/tcp, [::]:3307->3306/tcp						

The status bar at the bottom indicates the current file is 'index.js' in the 'src' directory, with 77 lines of code and 4 spaces. It also shows the active language is JavaScript and the spell checker is on.



Responses

Curl

```
curl -X 'GET' \
  'http://localhost:3000/health' \
  -H 'accept: */*'
```

Request URL

http://localhost:3000/health

Server response

Code Details

200

Response body

```
{
  "status": "ok",
  "db": "connected",
  "timestamp": "2026-03-20T02:41:21.269Z"
}
```

Response headers

```
connection: keep-alive
content-length: 71
content-type: application/json; charset=utf-8
date: Fri, 20 Mar 2026 02:41:21 GMT
etag: W/"47-Mw7OLnrI80XFNN/6wGF5PMwS2/Q"
keep-alive: timeout=5
x-powered-by: Express
```

Curl

```
curl -X 'GET' \
  'http://localhost:3000/api/users' \
  -H 'accept: application/json'
```

Request URL

http://localhost:3000/api/users

Server response

Code Details

200

Response body

```
{
  {
    "id": 1,
    "name": "Juan Pérez",
    "email": "juan.perez@email.com",
    "created_at": "2026-03-20T02:17:21.000Z"
  },
  {
    "id": 2,
    "name": "María García",
    "email": "maria.garcia@email.com",
    "created_at": "2026-03-20T02:17:21.000Z"
  },
  {
    "id": 3,
    "name": "Carlos López",
    "email": "carlos.lopez@email.com",
    "created_at": "2026-03-20T02:17:21.000Z"
  },
  {
    "id": 4,
    "name": "Ana Martínez",
    "email": "ana.martinez@email.com",
    "created_at": "2026-03-20T02:17:21.000Z"
  }
}
```

Response headers

```
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
  "name": "Ronald Llamas",
  "email": "ronaldllamas@gmail.com"
}'
```

Request URL

http://localhost:3000/api/users

Server response

Code	Details
------	---------

201

Response body

```
{
  "id": 1,
  "name": "Ronald Llamas",
  "email": "ronaldllamas@gmail.com"
}
```

Response headers

```
connection: keep-alive
content-length: 63
content-type: application/json; charset=utf-8
date: Fri, 20 Mar 2020 02:44:40 GMT
etag: W/"3f-S6MYXSmSpunApwUCL70zfvpgE"
keep-alive: timeout=5
x-powered-by: Express
```

Responses

Code	Description	Links
201	Usuario creado	No links
400	Datos faltantes	No links
409	Email duplicado	No links